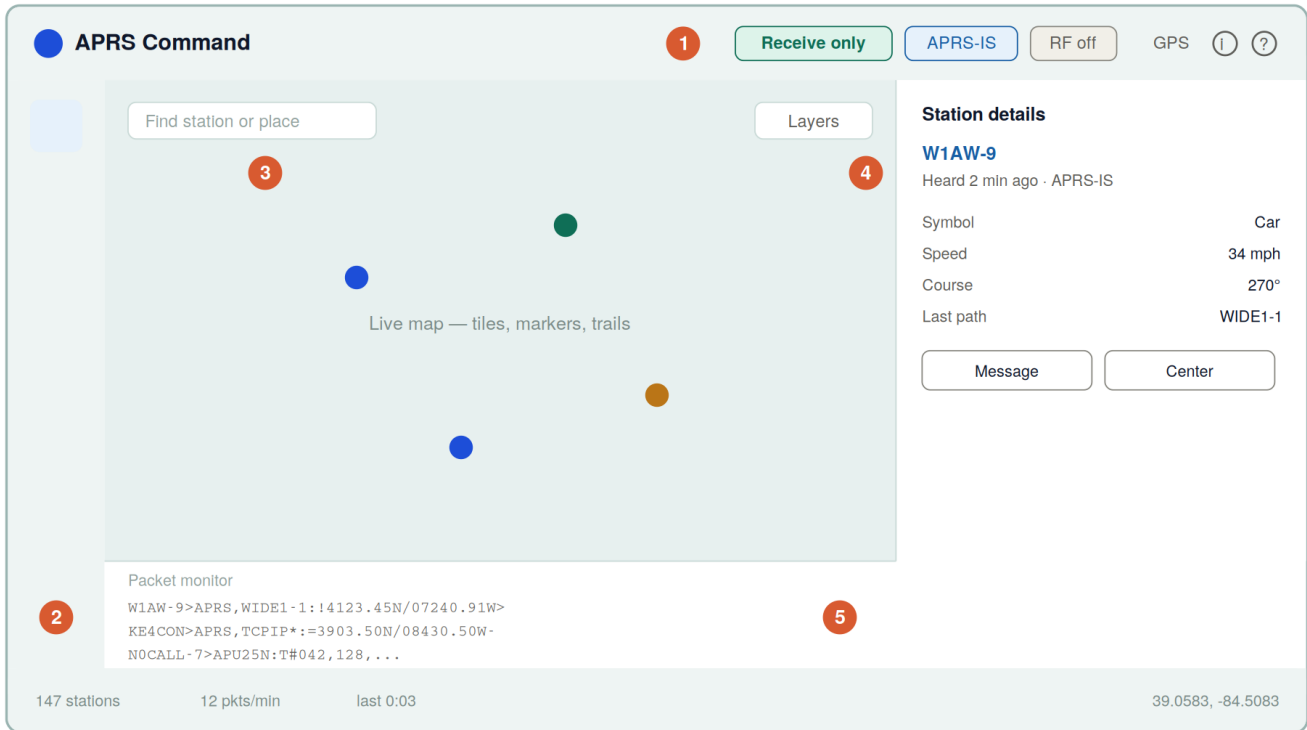


APRS Command — UI Design Proposal

A proposed cross-platform layout and the reasoning behind it. Notes space is included after each section.



Proposed layout. Numbered callouts map to the sections below.

- 1 Top status bar — connection + transmit state, always visible
- 2 Left icon rail — switch primary views (map, stations, messages, ...)
- 3 Map canvas — full-size workspace with search, layers, zoom
- 4 Right detail panel — contextual, collapsible/dockable
- 5 Packet monitor + status strip — collapsible, live telemetry

1 · 2 Let the map breathe; make panels dockable

Today everything competes for room in a fixed 2×2 grid, so the map, station list, packet monitor, and feature content are all permanently half-size. Give the map the whole canvas and turn everything else into panels you can show, hide, resize, or pop out — the way modern editors and GIS apps work. On a large monitor you spread out; on a Raspberry Pi touchscreen or a laptop in a vehicle you collapse to just map plus station list. Avalonia's docking library (Dock.Avalonia) gives full drag-rearrange docking; collapsible panels with splitters get most of the benefit with less work.

Notes

2 Left icon rail instead of crammed buttons

A vertical icon rail on the left scales better than a row of eight-plus buttons, stays out of the map's way, and is friendlier for touch. Use it for switching primary views, and reserve the top bar for global state and actions (connection, transmit, settings, help). "Where am I" on the left, "what is the system doing" on top — each area does one job. (This complements, rather than replaces, the top command bar we sketched earlier; you can keep both.)

Notes

1 Make transmit safety impossible to miss

Because the whole app is safe-by-default, the most important pixel is the transmit state. A prominent "Receive only" badge sits top-right; the moment RF or APRS-IS transmit is armed it should turn red and read "Transmit armed." Today those header chips are hardcoded text — making them live and color-coded is both a safety feature and the thing experienced operators glance at constantly.

Notes

5 A real, live status bar

The bottom strip — station count, packets per minute, last-heard age, cursor coordinates — is the at-a-glance telemetry UI-View operators relied on, and it costs almost nothing once the live data pipeline is connected.

Notes

Theming and a design-token system

Two practical moves. First, the current views hardcode hex colors inline everywhere; pulling those into one theme (colors, spacing, a type scale) makes the app consistent and far easier to restyle later. Second, ship a real dark mode plus a high-contrast / large-text mode — operators run these at night and in the field, and UI-View's night mode was well loved. Avalonia's Fluent theme provides light/dark almost for free.

Notes

Smaller modern touches

Worth considering once the core lands: map layer toggles (stations / objects / weather / trails / range rings); a search-and-center box; follow-me for your own GPS position; right-click context menus on stations; and a command palette (Ctrl-K) to jump to any station or feature. None are essential, but they are what make the app feel current rather than a port of a 2004 program.

Notes

Decisions to make

- Dockable map-first with a left rail, or the simpler top command bar?

- Dark mode and high-contrast mode — priority for v1 or later?

- Which smaller touches matter to you (layers, follow-me, command palette...)?

- Target device priority: desktop, laptop, or Raspberry Pi touchscreen first?

Free notes
